

Avviso per tutti i docenti e i responsabili dei corsi

A seguito di un importante aggiornamento della piattaforma, attualmente in corso, vi chiediamo di **non duplicare i quiz contenenti domande random** fino a nuova comunicazione.

Questa misura è necessaria per consentirci di completare l'aggiornamento e risolvere le problematiche riscontrate nel minor tempo possibile.

Vi ringraziamo per la collaborazione.



Ingegneria del Software

[Torna al corso](#)

Stato	Completato
Iniziato	giovedì, 25 giugno 2026, 10:11
Terminato	giovedì, 25 giugno 2026, 10:28
Tempo impiegato	17 min. 9 secondi
Punteggio	48,50/50,00
Valutazione	97,00 su un massimo di 100,00

Domanda 1

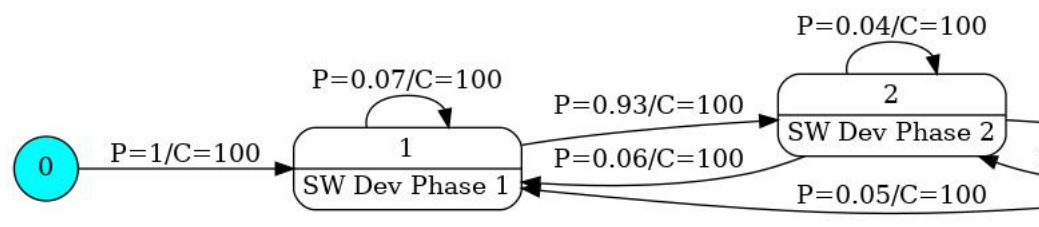
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 600.000000 ?

Scegli un'alternativa:

- ☒ 1. 0.0672 Correct.
- ☐ 2. 0.5336
- ☐ 3. 0.04704

Your answer is correct.

La risposta corretta è: 0.0672

Domanda 2

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile x_1 rappresenta lo stato precedente ad x e la variabile x_2 lo stato precedente ad x_1 .

Si consideri il seguente requisito:

Se il sistema è nello stato 1 allora o permane nello stato di 1 oppure passa allo stato 2.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☐ a. `assert((x1 == 1) ? ((x == 1) || (x == 2)) : 0)`
- ☒ b. `assert((x1 == 1) ? ((x == 1) || (x == 2)) : 1)` ✓
- ☐ c. `assert((x == 1) ? ((x1 == 1) || (x1 == 2)) : 1)`

Your answer is correct.

La risposta corretta è:

`assert((x1 == 1) ? ((x == 1) || (x == 2)) : 1)`

Domanda 3

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive l'obiettivo del "validity check" che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Assicurarsi che i requisiti funzionali descrivano tutte le funzionalità del sistema.
- ☒ 2. Assicurarsi che un sistema che soddisfa i requisiti risolve il problema del "customer". ✓
- ☐ 3. Assicurarsi che non ci siano requisiti in conflitto con altri requisiti.

Your answer is correct.

La risposta corretta è: Assicurarsi che un sistema che soddisfa i requisiti risolve il problema del "customer".

Domanda 4

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La WBS (Work Breakdown Structure) di un progetto consiste di 4 tasks: T0, T1, T2, T3.

I vincoli di precedenza tra tasks sono dati nella *tabella delle precedenze* definita nel seguito.

ID Successor Task	ID Predecessor Task
2	0
2	1
3	0
3	1

La riga i è nella tabella delle precedenze

La riga (i, j) nella tabella delle precedenze indica che il task j precede il task i (cioè che il task i può iniziare solo dopo che il task j è terminato).

L'effort (in PM, Person-Months) da dedicare a ciascun task per completare il progetto è dato dalla *tabella degli efforts* definita nel seguito:

Task ID	Effort
0	8
1	1
2	2
3	9

La riga (i, k) nella tabella degli efforts indica che il completamento del task i richiede un effort di k PM.

Lo scheduling per i tasks è definito attraverso una *tabella di scheduling*. Ciascuna riga della tabella di scheduling consiste di una tripla (i, j, k) dove:

1. i è il time slot (mese, nel caso in cui l'effort è misurato in PM);
2. j è il task
3. k è l'effort (positivo) dedicato al task j nel time slot i

Nella tabella di scheduling sono presenti solo le triple in cui l'effort è positivo. Per tutte le altre triple si intende che l'effort è 0.

Un esempio di *tabella di scheduling* è il seguente:

Time slot	Task	Effort (PM)
0	0	2
0	2	1
1	0	5
2	2	3

Nella tabella sopra al task T0 sono dedicati in totale 7 PM (2 nel time slot 0 e 5 nel time slot 1) ed al task T2 sono dedicati in totale 4 PM (1 nel time slot 0 e 3 nel time slot 2).

Nel time slot 0 l'effort totale è 3 (2PM su T0 ed 1 PM su T2)

Nel time slot 1 l'effort totale è 5 (5 PM su T0).

Nel time slot 2 l'effort totale è 3 (3 PM su T2).

Disponedo di 5 unità di personale (e quindi per ogni time slot l'effort totale può essere al più 5), quali delle seguenti tabelle di scheduling soddisfa i vincoli espressi dalle tabelle delle precedenze e degli efforts definite sopra?

Scegli un'alternativa:

- ☐ 1.

Time slot (Month)	Task	Effort (PM)
0	0	5
1	0	2
1	1	1
2	3	5
3	2	1
3	3	4
4	2	1
- ☒ 2.

Time slot (Month)	Task	Effort (PM)
0	0	5
1	0	3
1	1	1
2	3	5
3	2	1
3	3	4
4	2	1

 Correct.
- ☐ 3.

Time slot (Month)	Task	Effort (PM)
0	1	1
2	3	4
3	3	5
4	0	3
5	0	5
6	2	2

Your answer is correct.

La risposta corretta è:

Time slot (Month)	Task	Effort (PM)
0	0	5
1	0	3
1	1	1
2	3	5
3	2	1
3	3	4
4	2	1

Domanda 5

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile $x1$ rappresenta lo stato precedente ad x e la variabile $x2$ lo stato precedente ad $x1$.

Si consideri il seguente requisito:

Il valore assoluto della differenza tra il valore dello stato presente e di quello precedente è al più 2.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☐ a. `assert( abs(x - x1) >= 2)`
- ☒ b. `assert( abs(x - x1) <= 2)` ☺
- ☐ c. `assert( (x - x1) <= 2)`

Your answer is correct.

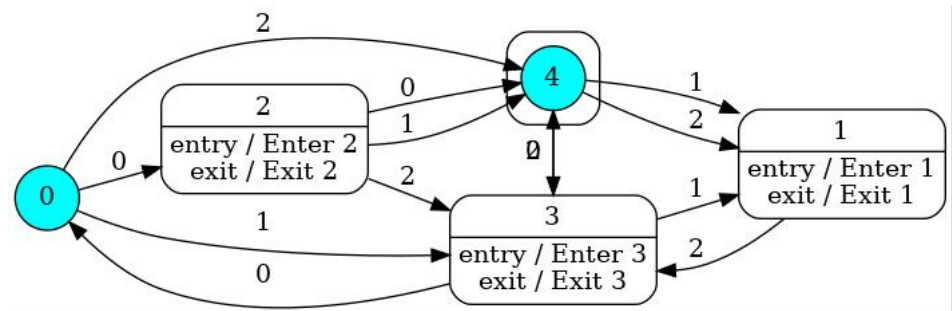
La risposta corretta è:
`assert( abs(x - x1) <= 2)`

Domanda 6

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri lo state diagram in figura.



Quale delle seguenti funzioni di transizione meglio lo rappresenta?

Scegli un'alternativa:

☐ 1.

```
#include "main.h"

int f(int x, int u)
{ switch (x) {
  case 0: switch (u) {
    case 0: return (3);
    case 1: return (1);
    case 2: return (0);
    default: return (0); }
  case 1: switch (u) {
    case 0: return (3);
    case 1: return (3);
    case 2: return (2);
    default: return (0); }
  case 2: switch (u) {
    case 0: return (1);
    case 1: return (2);
    case 2: return (1);
    default: return (0); }
  case 3: switch (u) {
    case 0: return (0);
    case 1: return (3);
    case 2: return (3);
    default: return (0); }
  case 4: switch (u) {
    case 0: return (0);
    case 1: return (1);
    case 2: return (4);
    default: return (0); }
  default: return (0);
} } // f()
```

☒ 2.

```
#include "main.h"

int f(int x, int u)
{ switch (x) {
  case 0: switch (u) {
    case 0: return (2);
    case 1: return (3);
    case 2: return (4);
    default: return (0); }
  case 1: switch (u) {
    case 0: return (1);
    case 1: return (1);
    case 2: return (3);
    default: return (0); }
  case 2: switch (u) {
    case 0: return (4);
    case 1: return (4);
    case 2: return (3);
    default: return (0); }
  case 3: switch (u) {
    case 0: return (0);
    case 1: return (1);
    case 2: return (4);
    default: return (0); }
  case 4: switch (u) {
    case 0: return (3);
    case 1: return (1);
    case 2: return (1);
    default: return (0); }
  default: return (0);
} } // f()
```

Correct.

☐ 3.

```

-- -- #include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (3);
        case 1: return (1);
        case 2: return (3);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (3);
        case 1: return (2);
        case 2: return (4);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (3);
        case 1: return (1);
        case 2: return (3);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (4);
        case 1: return (4);
        case 2: return (4);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (1);
        case 1: return (3);
        case 2: return (1);
        default: return (0); }
    default: return (0);
}
} // f()

```

Your answer is correct.

```

#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (2);
        case 1: return (3);
        case 2: return (4);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (1);
        case 1: return (1);
        case 2: return (3);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (4);
        case 1: return (4);
        case 2: return (3);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (0);
        case 1: return (1);
        case 2: return (4);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (3);
        case 1: return (1);
        case 2: return (1);
        default: return (0); }
    default: return (0);
}
} // f()

```

La risposta corretta è:

Domanda 7

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive il criterio di "requirements verifiability" che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Per ciascuna componente del sistema, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che essa soddisfa tutti i requisiti.
- ☒ 2. Per ciascun requisito, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che il sistema sviluppato soddisfa il requisito considerato. ✓
- ☐ 3. Per ciascuna coppia di componenti, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che l'interazione tra le componenti soddisfa tutti i requisiti di interfaccia.

Your answer is correct.

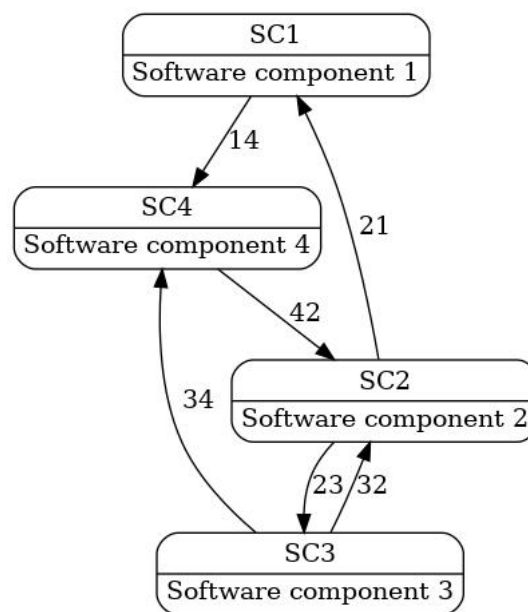
La risposta corretta è: Per ciascun requisito, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che il sistema sviluppato soddisfa il requisito considerato.

Domanda 8

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri la seguente architettura software:



Quale dei seguenti frammenti di pseudo codice meglio la rappresenta?

Scegli un'alternativa:

- ☒ 1. // Begin input/output connections for system components  Correct.
- Input 14 of SC4 = Output 14 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 23 of SC3 = Output 23 of SC2
Input 32 of SC2 = Output 32 of SC3
Input 34 of SC4 = Output 34 of SC3
Input 42 of SC2 = Output 42 of SC4
// End input/output connections for system components
- ☐ 2. // Begin input/output connections for system components
- Input 13 of SC3 = Output 13 of SC1
Input 14 of SC4 = Output 14 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 23 of SC3 = Output 23 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 41 of SC1 = Output 41 of SC4
Input 42 of SC2 = Output 42 of SC4
// End input/output connections for system components
- ☐ 3. // Begin input/output connections for system components
- Input 12 of SC2 = Output 12 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 41 of SC1 = Output 41 of SC4
Input 43 of SC3 = Output 43 of SC4
// End input/output connections for system components

Your answer is correct.

La risposta corretta è: // Begin input/output connections for system components

Input 14 of SC4 = Output 14 of SC1

Input 21 of SC1 = Output 21 of SC2

Input 23 of SC3 = Output 23 of SC2

Input 32 of SC2 = Output 32 of SC3

Input 34 of SC4 = Output 34 of SC3

Input 42 of SC2 = Output 42 of SC4

// End input/output connections for system components

Domanda 9

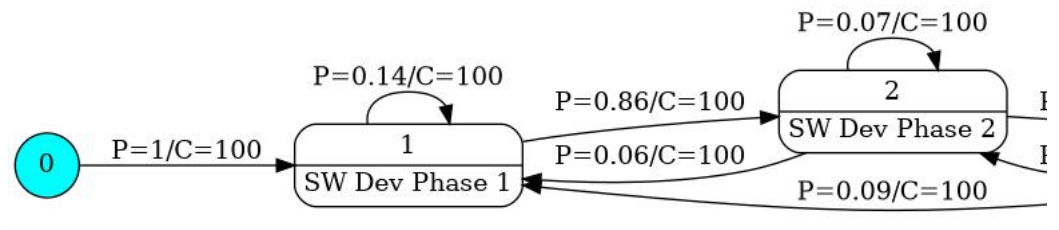
Risposta errata

Punteggio ottenuto -0,50 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 600.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.1061
- ☒ 2. 0.07427 ✗ Incorrect.
- ☐ 3. 0.55305

Your answer is incorrect.

La risposta corretta è: 0.1061

Domanda 10

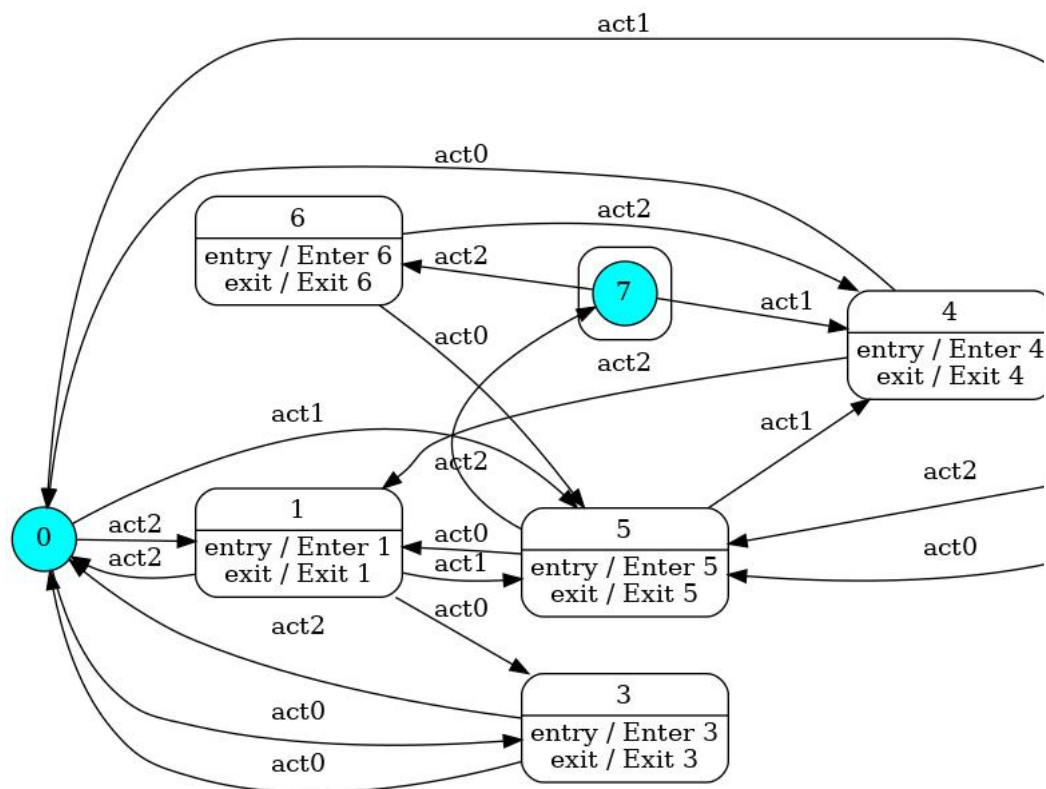
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La transition coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di transizioni (archi nel grafo dello state diagram) percorsi almeno una volta.

Si consideri lo state diagram in figura:

Si consideri lo state diagram UML in figura:



ed il seguente insieme di test cases:

Test case 1: act0 act2 act1 act1 act2

Test case 2: act2 act1 act0 act1 act2

Test case 3: act2 act2 act2 act0

Quale delle seguenti è la migliore stima della transition coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☐ 1. Transition coverage: 0.630
- ☐ 2. Transition coverage: 0.246
- ☒ 3. Transition coverage: 0.333 ✓ Correct.

Your answer is correct.

La risposta corretta è: Transition coverage: 0.333

Domanda 11

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive l'obiettivo del "check di consistenza" che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Assicurarsi che i requisiti funzionali descrivano tutte le funzionalità del sistema.
- ☒ 2. Assicurarsi che non ci siano requisiti in conflitto con altri requisiti. ✓
- ☐ 3. Assicurarsi che per ogni requisito esista un insieme di test che lo possa verificare.

Your answer is correct.

La risposta corretta è: Assicurarsi che non ci siano requisiti in conflitto con altri requisiti.

Domanda 12

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile x1 rappresenta lo stato precedente ad x e la variabile x2 lo stato precedente ad x1.

Si consideri il seguente requisito:

Se lo stato vale 0 per due transizioni consecutive, allora alla transizione successiva lo stato avrà valore 1.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☐ a. `assert( (x == 0) && (x1 == 0) ? (x2 == 1) : 1 )`
- ☒ b. `assert( (x2 == 0) && (x1 == 0) ? (x == 1) : 1 )` ✓
- ☐ c. `assert( (x2 == 0) && (x1 == 0) ? (x == 1) : 0 )`

Your answer is correct.

La risposta corretta è:
`assert( (x2 == 0) && (x1 == 0) ? (x == 1) : 1 )`

Domanda 13

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile $x1$ rappresenta lo stato precedente ad x e la variabile $x2$ lo stato precedente ad $x1$.

Si consideri il seguente requisito:

La differenza tra il valore dello stato presente e di quello precedente è compresa tra -1 ed 1.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☐ a. `assert( (-1 <= (x - x1)) && ((x1 - x) <= 1) )`
- ☐ b. `assert( ( x - x1) <= 1 )`
- ☒ c. `assert( (-1 <= (x - x1)) && ((x - x1) <= 1) )` 

Your answer is correct.

La risposta corretta è:

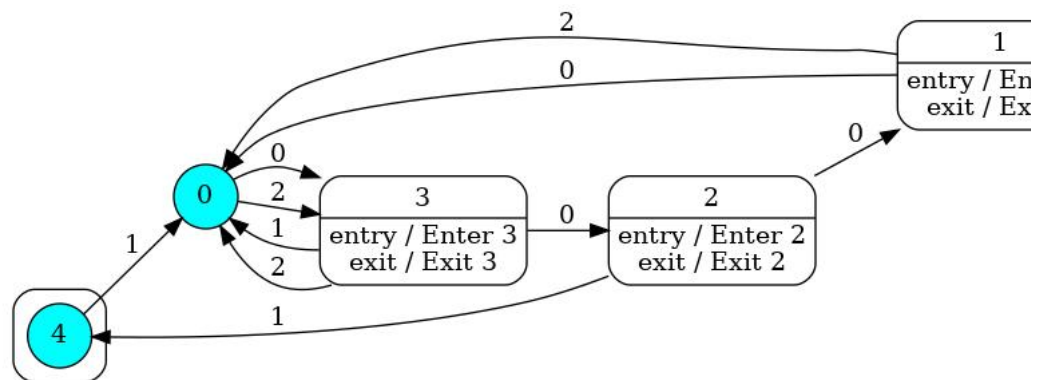
`assert( (-1 <= (x - x1)) && ((x - x1) <= 1) )`

Domanda 14

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri lo state diagram in figura.



Quale delle seguenti funzioni di transizione meglio lo rappresenta?

Scegli un'alternativa

Scelgi un'alternativa.

- ☒ 1. `#include "main.h"` ✔ Correct.

```
int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (3);
        case 1: return (0);
        case 2: return (3);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (0);
        case 1: return (1);
        case 2: return (0);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (1);
        case 1: return (4);
        case 2: return (2);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (2);
        case 1: return (0);
        case 2: return (0);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (4);
        case 1: return (0);
        case 2: return (4);
        default: return (0); }
    default: return (0);
}
} // f()
```

- ☐ 2. `#include "main.h"`

```
int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (0);
        case 1: return (2);
        case 2: return (4);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (3);
        case 1: return (2);
        case 2: return (1);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (4);
        case 1: return (4);
        case 2: return (0);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (1);
        case 1: return (1);
        case 2: return (1);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (2);
        case 1: return (0);
        case 2: return (1);
        default: return (0); }
    default: return (0);
}
} // f()
```

```

3. #include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (2);
        case 1: return (2);
        case 2: return (0);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (4);
        case 1: return (4);
        case 2: return (3);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (4);
        case 1: return (4);
        case 2: return (1);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (0);
        case 1: return (2);
        case 2: return (3);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (4);
        case 1: return (3);
        case 2: return (3);
        default: return (0); }
    default: return (0);
}
} // f()

```

Your answer is correct.

```

#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (3);
        case 1: return (0);
        case 2: return (3);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (0);
        case 1: return (1);
        case 2: return (0);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (1);
        case 1: return (4);
        case 2: return (2);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (2);
        case 1: return (0);
        case 2: return (0);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (4);
        case 1: return (0);
        case 2: return (4);
        default: return (0); }
    default: return (0);
}
} // f()

```

La risposta corretta è:

Domanda 15

Si consideri il seguente frammento di pseudo codice.

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

// Begin input/output connections for system components

Input 12 of SC2 = Output 12 of SC1

Input 13 of SC3 = Output 13 of SC1

Input 23 of SC3 = Output 23 of SC2

Input 24 of SC4 = Output 24 of SC2

Input 31 of SC1 = Output 31 of SC3

Input 32 of SC2 = Output 32 of SC3

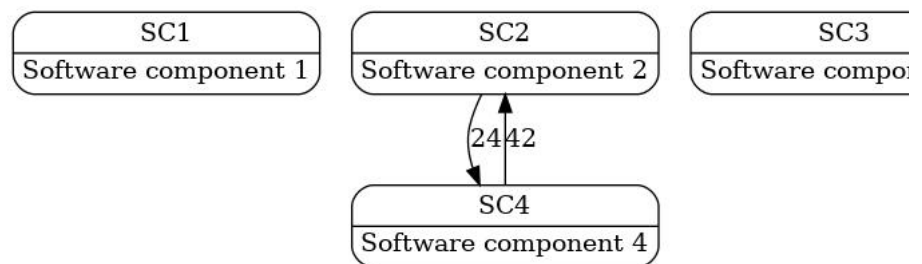
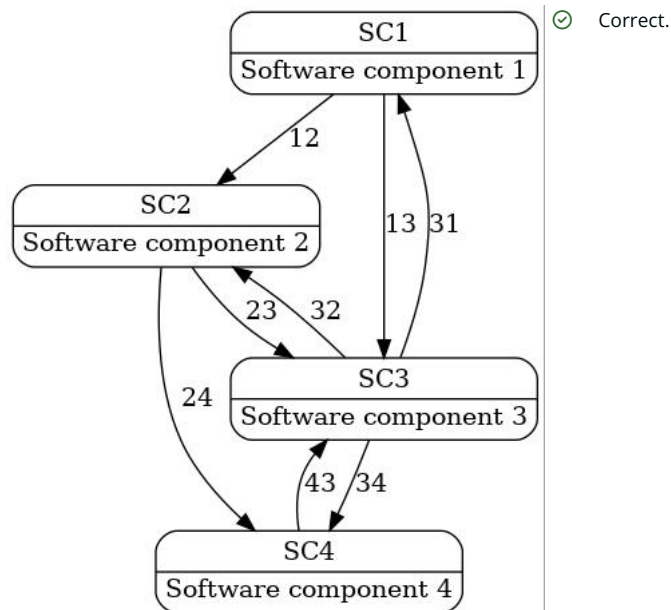
Input 34 of SC4 = Output 34 of SC3

Input 43 of SC3 = Output 43 of SC4

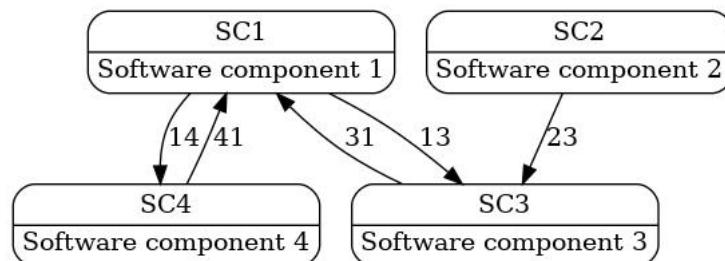
// End input/output connections for system components

Quale delle seguenti architetture software meglio lo rappresenta ?

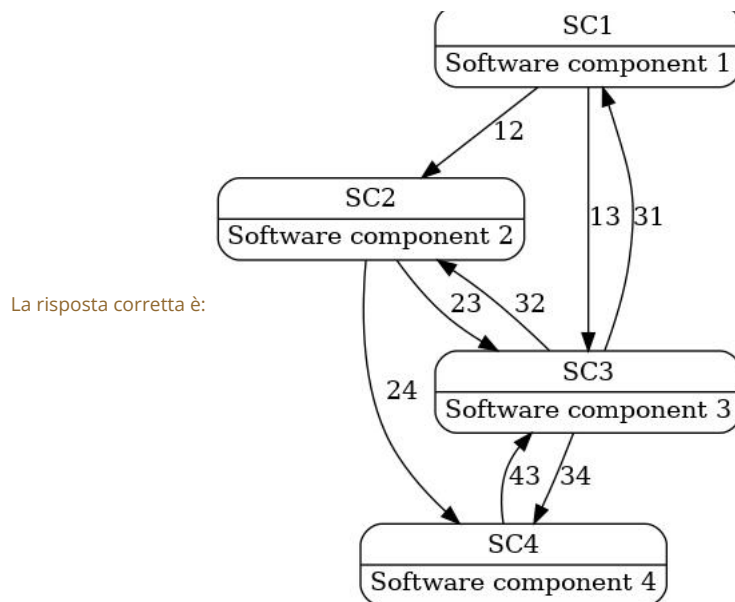
Scegli un'alternativa:

☐ 1.☒ 2.

✔ Correct.

☐ 3.

Your answer is correct.



Domanda 16

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La WBS (Work Breakdown Structure) di un progetto consiste di 4 tasks: T0, T1, T2, T3.

I vincoli di precedenza tra tasks sono dati nella *tabella delle precedenze* definita nel seguito.

ID Successor Task	ID Predecessor Task
1	0
2	0
3	1
3	2

La riga (i, j) nella tabella delle precedenze indica che il task j precede il task i (cioè che il task i può iniziare solo dopo che il task j è terminato).

L'effort (in PM, Person-Months) da dedicare a ciascun task per completare il progetto è dato dalla *tabella degli efforts* definita nel seguito:

Task ID	Effort
0	4
1	5
2	5
3	2

La riga (i, k) nella tabella degli efforts indica che il completamento del task i richiede un effort di k PM.

Lo scheduling per i tasks è definito attraverso una *tabella di scheduling*. Ciascuna riga della tabella di scheduling consiste di una tripla (i, j, k) dove:

1. i è il time slot (mese, nel caso in cui l'effort è misurato in PM);
2. j è il task

3. k è l'effort (positivo) dedicato al task j nel time slot i

Nella tabella di scheduling sono presenti solo le triple in cui l'effort è positivo. Per tutte le altre triple si intende che l'effort è 0.

Un esempio di *tabella di scheduling* è il seguente:

Time slot	Task	Effort (PM)
0	0	2
0	2	1
1	0	5
2	2	3

Nella tabella sopra al task T0 sono dedicati in totale 7 PM (2 nel time slot 0 e 5 nel time slot 1) ed al task T2 sono dedicati in totale 4 PM (1 nel time slot 0 e 3 nel time slot 2).

Nel time slot 0 l'effort totale è 3 (2PM su T0 ed 1 PM su T2)

Nel time slot 1 l'effort totale è 5 (5 PM su T0).

Nel time slot 2 l'effort totale è 3 (3 PM su T2).

Disponedo di 5 unità di personale (e quindi per ogni time slot l'effort totale può essere al più 5), quali delle seguenti tabelle di scheduling soddisfa i vincoli espressi dalle tabelle delle precedenze e degli efforts definite sopra?

Scegli un'alternativa:

- ☐ 1.

Time slot (Month)	Task	Effort (PM)
0	0	4
2	1	5
3	2	4
4	3	2
- ☒ 2.

Time slot (Month)	Task	Effort (PM)
0	0	4
2	1	5
3	2	5
4	3	2

 Correct.
- ☐ 3.

Time slot (Month)	Task	Effort (PM)
0	2	5
2	0	4
4	1	5
7	3	2

Your answer is correct.

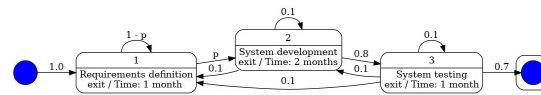
La risposta corretta è:

Time slot (Month)	Task	Effort (PM)
0	0	4
2	1	5
3	2	5
4	3	2

Domanda 17

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri il processo software in figura

Qual'è la migliore stima per il valore minimo di p che garantisce che la probabilità che il processo termini in esattamente 4 mesi sia almeno 0.5 ?

Scegli un'alternativa:

- ☐ 1. 0.959
- ☐ 2. 0.645
- ☒ 3. 0.893 ✓

Your answer is correct.

La risposta corretta è: 0.893

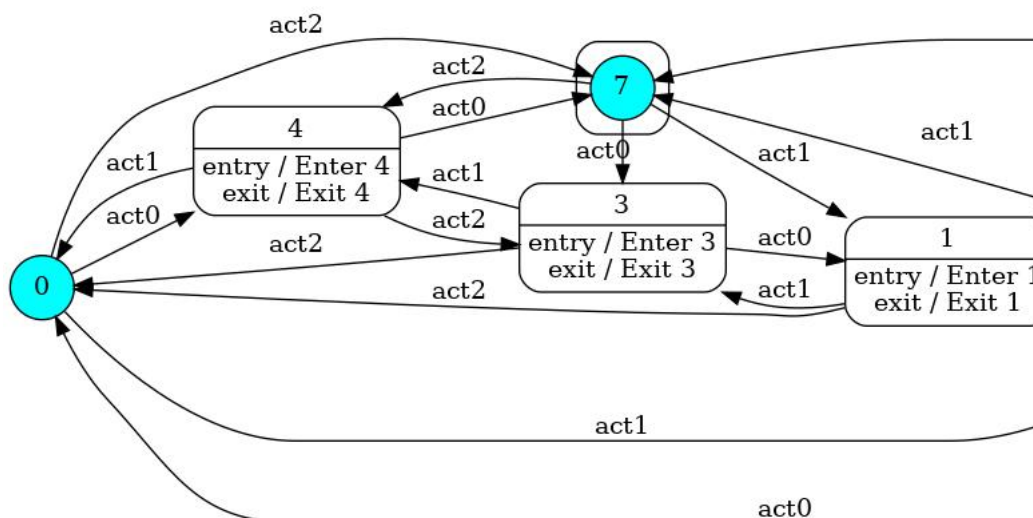
Domanda 18

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La **state coverage** di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di stati (inclusi START ed END) raggiunti almeno una volta.

Si consideri lo state diagram UML in figura:



Si consideri il seguente insieme di test cases:

Test case 1: act0 act2 act2 act1 act2

Test case 2: act0 act0

Test case 3: act1 act0 act1

Quale delle seguenti è la migliore stima della state coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☐ 1. State coverage: 0.938
- ☐ 2. State coverage: 0.612
- ☒ 3. State coverage: 0.875 Correct.

Your answer is correct.

La risposta corretta è: State coverage: 0.875

Domanda 19

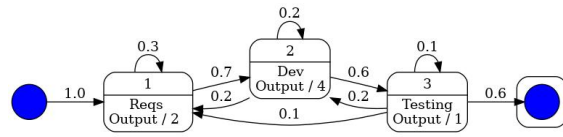
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software di n stati può essere modellato con una coppia (A, Y) dove:

- A è una matrice che definisce una catena di Markov con n stati etichettati con interi da 0 ad $n-1$. Lo stato iniziale è etichettato con 0 mentre lo stato finale con $n-1$.
- Y è un vettore di n elementi che definisce l'output della catena di Markov associando un output (se definito) a ciascuno stato.

Quale delle seguenti rappresentazioni (A, Y) meglio modella il processo software in figura ?



Scegli un'alternativa:

- ☒ 1. $A[0][1] = 1.0$ ✓

$A[1][1] = 0.3$

$A[1][2] = 0.7$

$A[2][1] = 0.2$

$A[2][2] = 0.2$

$A[2][3] = 0.6$

$A[3][1] = 0.1$

$A[3][2] = 0.2$

$A[3][3] = 0.1$

$A[3][4] = 0.6$

$A[4][4] = 1.0$

$Y[0] = -$

$Y[1] = 2$

$Y[2] = 4$

$Y[3] = 1$

$Y[4] = -$

- ☐ 2. $A[0][1] = 1.0$

$A[1][1] = 0.3$

$A[1][2] = 0.7$

$A[2][1] = 0.2$

$A[2][2] = 0.2$

$A[2][3] = 0.6$

$A[3][1] = 0.1$

$A[3][2] = 0.2$

$A[3][3] = 0.1$

$A[3][4] = 0.6$

$A[4][4] = 1.0$

$Y[0] = -$

$Y[1] = 2$

$Y[2] = 2$

$Y[3] = 1$

$Y[4] = -$

- ☐ 3. $A[0][1] = 1.0$
 $A[1][1] = 0.4$
 $A[1][2] = 0.6$
 $A[2][1] = 0.1$
 $A[2][2] = 0.1$
 $A[2][3] = 0.8$
 $A[3][1] = 0.1$
 $A[3][2] = 0.2$
 $A[3][3] = 0.1$
 $A[3][4] = 0.6$
 $A[4][4] = 1.0$

$Y[0] = -$
 $Y[1] = 2$
 $Y[2] = 4$
 $Y[3] = 1$
 $Y[4] = -$

Your answer is correct.

La risposta corretta è:

$A[0][1] = 1.0$
 $A[1][1] = 0.3$
 $A[1][2] = 0.7$
 $A[2][1] = 0.2$
 $A[2][2] = 0.2$
 $A[2][3] = 0.6$
 $A[3][1] = 0.1$
 $A[3][2] = 0.2$
 $A[3][3] = 0.1$
 $A[3][4] = 0.6$
 $A[4][4] = 1.0$
 $Y[0] = -$

$Y[1] = 2$
 $Y[2] = 4$
 $Y[3] = 1$
 $Y[4] = -$

Domanda 20

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quali delle seguenti attività può contribuire a validare i requisiti di un sistema ?

Scegli un'alternativa:

- ☐ 1. Costruire un prototipo e testarlo a fondo per evidenziare subito errori di implementazione.
- ☐ 2. Costruire un prototipo e valutarne attentamente le performance.
- ☒ 3. Costruire un prototipo, metterlo in esercizio ed accertarsi che i porti i benefici attesi. ✓

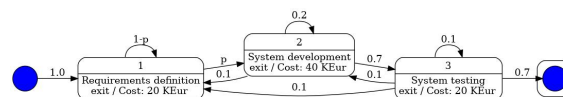
Your answer is correct.

La risposta corretta è: Costruire un prototipo, metterlo in esercizio ed accertarsi che i porti i benefici attesi.

Domanda 21

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri il processo software in figura**Qual'è la migliore stima per il valore minimo di p che garantisce che la probabilità che il processo termini con un costo di esattamente 80K EUR sia almeno 0.5.**

Scegli un'alternativa:

- ☐ 1. 0.785
- ☒ 2. 0.98 ✓
- ☐ 3. 0.538

Your answer is correct.

La risposta corretta è: 0.98

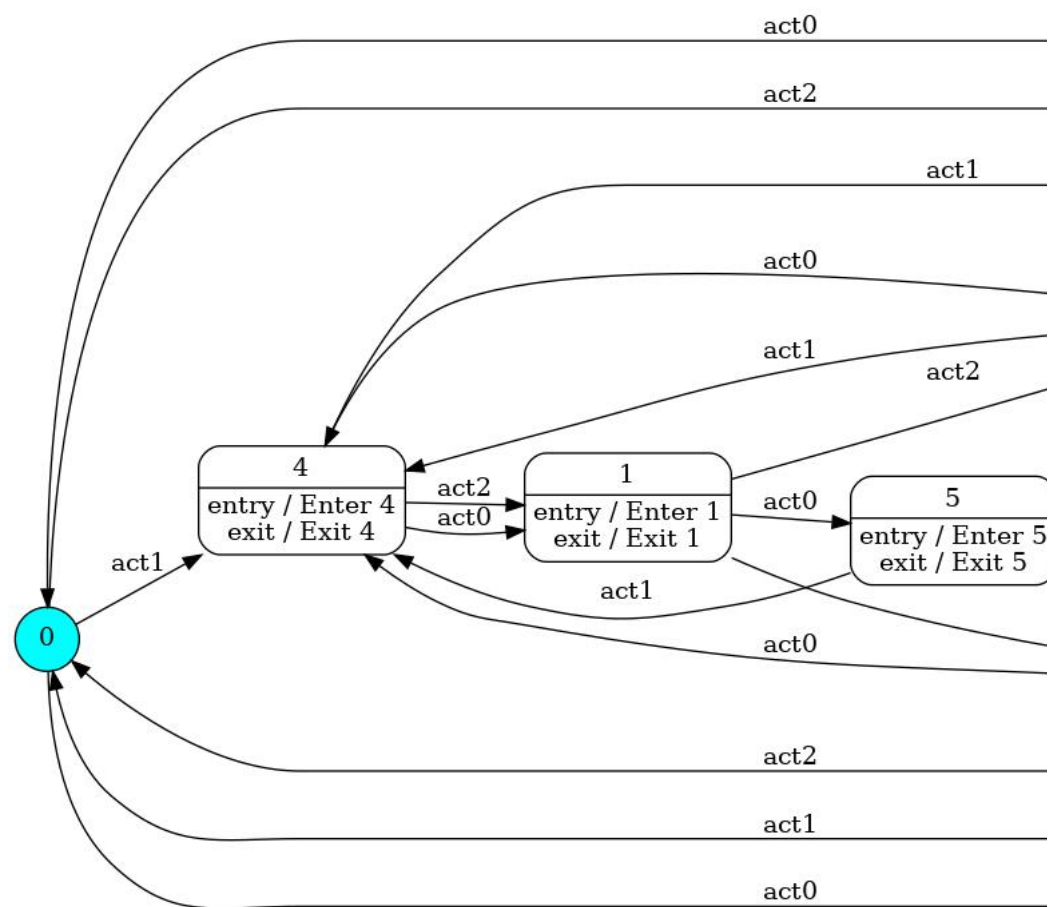
Domanda 22

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La state coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di stati (inclusi START ed END) raggiunti almeno una volta.

Si consideri lo state diagram UML in figura:



Si consideri il seguente insieme di test cases:

Test case 1: act2

Test case 2: act0 act2 act1 act0 act1

Test case 3: act0 act0

Quale delle seguenti è la migliore stima della state coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☐ 1. State coverage: 0.812
- ☒ 2. State coverage: 0.625 Correct.
- ☐ 3. State coverage: 0.438

Your answer is correct.

La risposta corretta è: State coverage: 0.625

Domanda 23

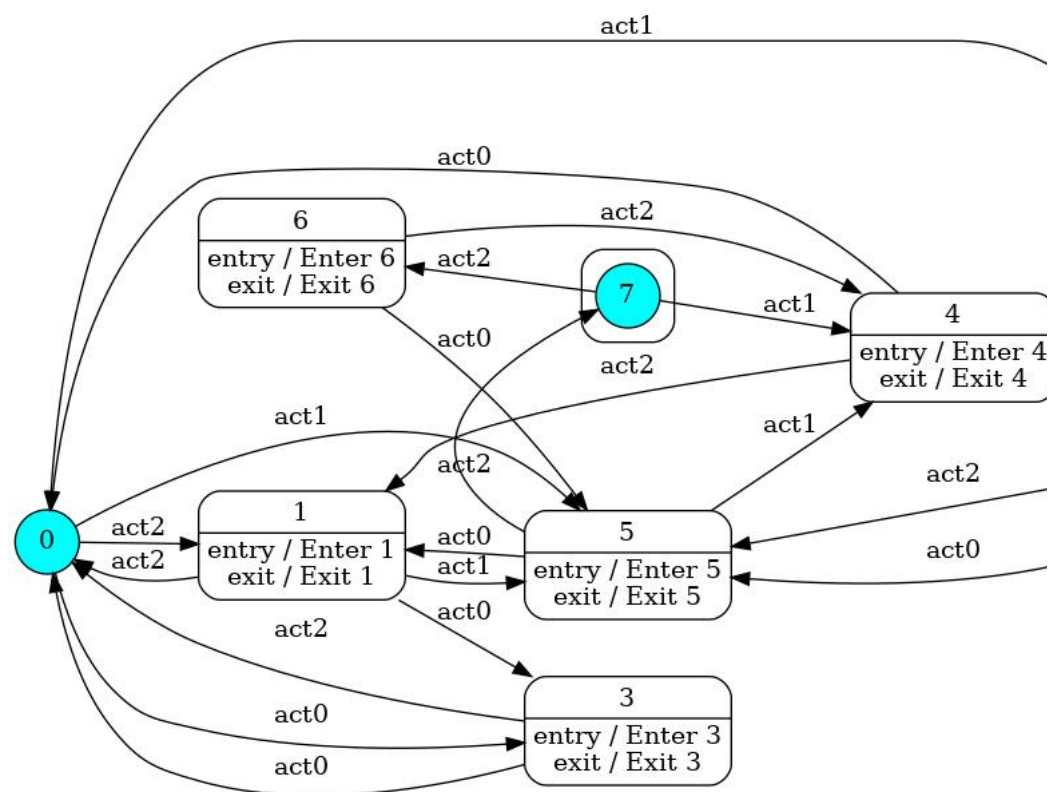
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La transition coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di transizioni (archi nel grafo dello state diagram) percorsi almeno una volta.

Si consideri lo state diagram in figura:

Si consideri lo state diagram UML in figura:



ed il seguente insieme di test cases:

Test case 1: act0 act2 act1 act1 act2

Test case 2: act2 act1 act0 act1 act2

Test case 3: act2 act2 act2 act0

Quale delle seguenti è la migliore stima della transition coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☒ 1. Transition coverage: 0.333 Correct.
- ☐ 2. Transition coverage: 0.630
- ☐ 3. Transition coverage: 0.246

Your answer is correct.

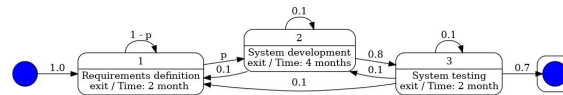
La risposta corretta è: Transition coverage: 0.333

Domanda 24

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri il processo software in figura



Qual'è la migliore stima per il valore minimo di p che garantisce che la probabilità che il processo termini in esattamente 8 mesi sia almeno 0.5 ?

Scegli un'alternativa:

- ☒ 1. 0.893 ✓
- ☐ 2. 0.959
- ☐ 3. 0.645

Your answer is correct.

La risposta corretta è: 0.893

Domanda 25

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo di sviluppo agile consiste di 3 iterazioni identiche di costo A . Alla fine di ogni iterazione vengono prese in considerazione le "change requests" e, se ve ne sono, l'iterazione viene ripetuta. Sia p la probabilità che ci siano "change requests" all fine di una iterazione. Il valore atteso del costo del progetto è:

Scegli un'alternativa:

- ☒ 1. $3 \cdot (1 + p) \cdot A$ ✓
- ☐ 2. $3 \cdot p \cdot A$
- ☐ 3. $3 \cdot (A + p)$

Your answer is correct.

La risposta corretta è: $3 \cdot (1 + p) \cdot A$

Domanda 26

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un *Path* in un programma è una sequenza di istruzioni dall'inizio alla fine del programma.

La *Path Coverage* di un insieme di test cases è la percentuale di path del programma che sono attraversati da almeno un test case.

Considerando la seguente funzione C, qual'è la path coverage con i tests cases dati.

```
int f(int x[3])
{
    if ((0)*x[0] + (-3)*x[1] + (3)*x[2] <= 2)
        if ((2)*x[0] + (-2)*x[1] + (4)*x[2] <= -2)
            {return (0); }
        else
            {return (1); }
    else if ((1)*x[0] + (-4)*x[1] + (0)*x[2] <= -4)
        if ((4)*x[0] + (3)*x[1] + (-1)*x[2] <= -3)
            {return (2); }
        else
            {return (3); }
    else
        {return (4); }
} /* f() */
```

Test 1: x[0] = 1, x[1] = -1, x[2] = 4.

Test 2: x[0] = 1, x[1] = 3, x[2] = 1.

Test 3: x[0] = -4, x[1] = 3, x[2] = -2.

Scegli un'alternativa:

- ☐ 1. Coverage: 0.794
- ☐ 2. Coverage: 0.390
- ☒ 3. Coverage: 0.600  Correct.

Your answer is correct.

La risposta corretta è: Coverage: 0.600

Domanda 27

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

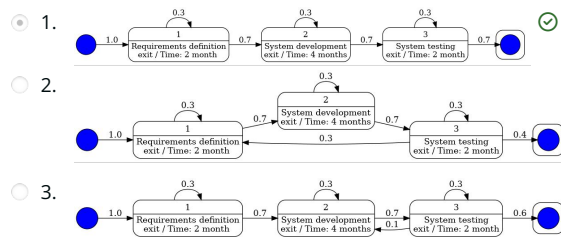
Un processo software consiste delle seguenti fasi:

- 1: Requirements definition,**
- 2: System development,**
- 3: System testing.**
- 4: Delivey (fase finale)**

La probabilità di passare da una fase i ($i= 1, 2, 3$) alla fase successiva ($i+1$) è sempre 0.7, mentre con probabilità 0.3 è necessario ripetere la fase in corso a causa di errori.

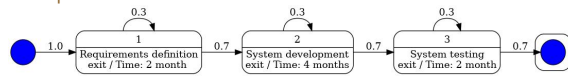
Quale delle seguenti catene di Markov meglio modella il processo software di cui sopra ?

Scegli un'alternativa:



Your answer is correct.

La risposta corretta è:



Domanda 28

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri la seguente funzione di transizione.

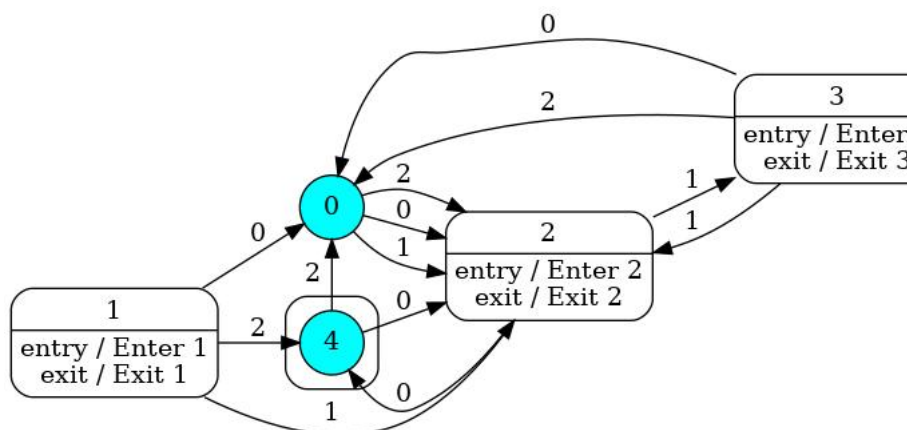
```
#include "main.h"

int f(int x, int u)
{ switch (x) {
  case 0: switch (u) {
    case 0: return (2);
    case 1: return (2);
    case 2: return (2);
    default: return (0); }
  case 1: switch (u) {
    case 0: return (0);
    case 1: return (2);
    case 2: return (4);
    default: return (0); }
  case 2: switch (u) {
    case 0: return (4);
    case 1: return (3);
    case 2: return (2);
    default: return (0); }
  case 3: switch (u) {
    case 0: return (0);
    case 1: return (2);
    case 2: return (0);
    default: return (0); }
  case 4: switch (u) {
    case 0: return (2);
    case 1: return (4);
    case 2: return (0);
    default: return (0); }
  default: return (0); }
} // f()
```

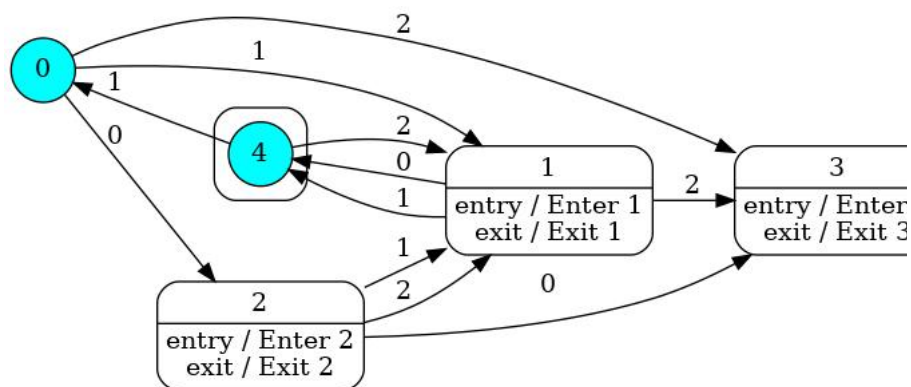
Quale dei seguenti UML state diagram rappresenta correttamente la macchina a stati finiti definita dalla funzione di transizione definita sopra?

Scegli un'alternativa:

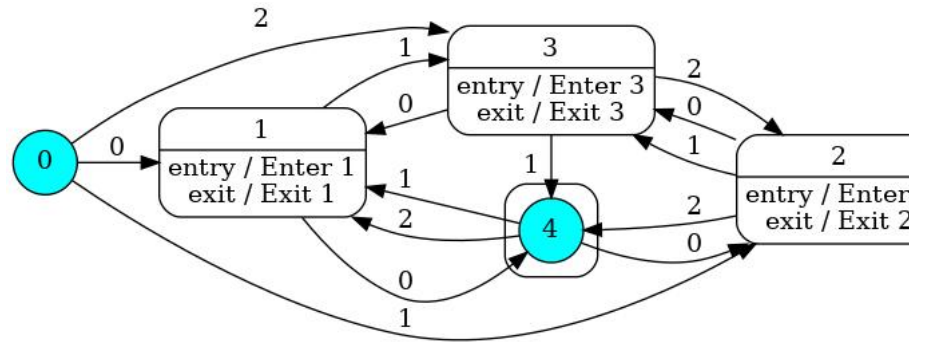
☒ 1.



☐ 2.

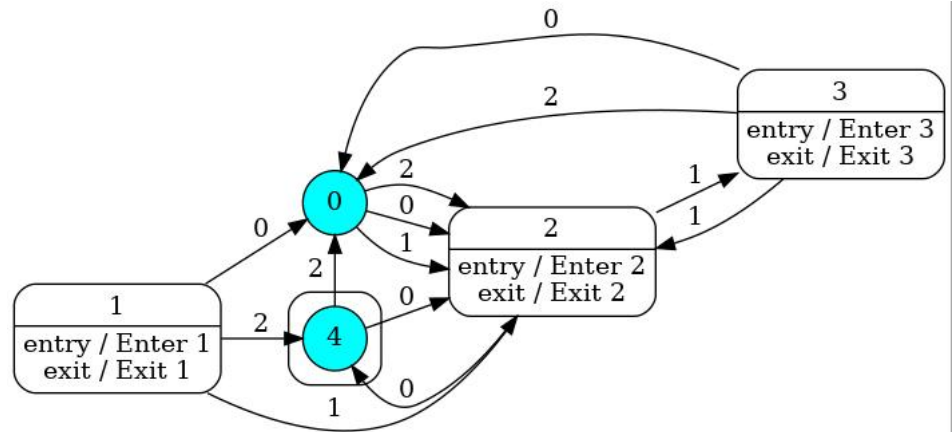


3.



Your answer is correct.

La risposta corretta è:

**Domanda 29**

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta un vettore di n elementi $x[0], \dots, x[n-1]$.

Si consideri il seguente requisito:

Il vettore x è ordinato con valori non decrescenti.

Quale dei seguenti monitors meglio definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☐ a. `for (i=0; i < n; i++) assert(x[i] <= x[i+1]);`
- ☒ b. `for (i=0; i < n-1; i++) assert(x[i] <= x[i+1]);` ✓
- ☐ c. `for (i=0; i < n-1; i++) assert(x[i] > x[i+1]);`

Your answer is correct.

La risposta corretta è:

`for (i=0; i < n-1; i++) assert(x[i] <= x[i+1]);`

Domanda 30

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un *Path* in un programma è una sequenza di istruzioni dall'inizio alla fine del programma.

La *Path Coverage* di un insieme di test cases è la percentuale di path del programma che sono attraversati da almeno un test case.

Considerando la seguente funzione C, qual'è la path coverage con i tests cases dati.

```
int f(int x[3])
{
    if ((3)*x[0] + (-1)*x[1] + (2)*x[2] <= 1)
        if ((2)*x[0] + (-3)*x[1] + (-1)*x[2] <= 2)
            {return (0); }
        else
            {return (1); }
    else if ((4)*x[0] + (4)*x[1] + (4)*x[2] <= -1)
        if ((-1)*x[0] + (4)*x[1] + (-1)*x[2] <= 3)
            {return (2); }
        else
            {return (3); }
    else
        {return (4); }
} /* f() */
```

Test 1: x[0] = 3, x[1] = -3, x[2] = -1.

Test 2: x[0] = -1, x[1] = 4, x[2] = -2.

Test 3: x[0] = 2, x[1] = 0, x[2] = -2.

Test 4: x[0] = -4, x[1] = -3, x[2] = -2.

Scegli un'alternativa:

- ☐ 1. Coverage: 0.892
- ☒ 2. Coverage: 0.800  Correct.
- ☐ 3. Coverage: 0.565

Your answer is correct.

La risposta corretta è: Coverage: 0.800

Domanda 31

Quale dei seguenti è un requisito non funzionale?

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Scegli un'alternativa:

- ☐ 1. L'output della funzione è un vettore della stessa dimensione del vettore di input.
- ☐ 2. L'output della funzione è un vettore ordinato in modo ascendente.
- ☒ 3. L'esecuzione della funzione occupa al più 2MB di RAM. ✔ Correct.

Your answer is correct.

La risposta corretta è: L'esecuzione della funzione occupa al più 2MB di RAM.

Domanda 32

Si consideri il seguente frammento di pseudo codice.

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

// Begin input/output connections for system components

Input 12 of SC2 = Output 12 of SC1

Input 13 of SC3 = Output 13 of SC1

Input 21 of SC1 = Output 21 of SC2

Input 23 of SC3 = Output 23 of SC2

Input 24 of SC4 = Output 24 of SC2

Input 31 of SC1 = Output 31 of SC3

Input 34 of SC4 = Output 34 of SC3

Input 41 of SC1 = Output 41 of SC4

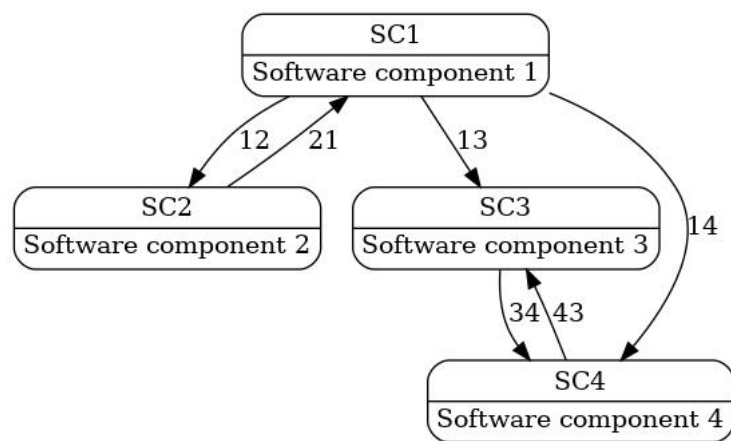
Input 43 of SC3 = Output 43 of SC4

// End input/output connections for system components

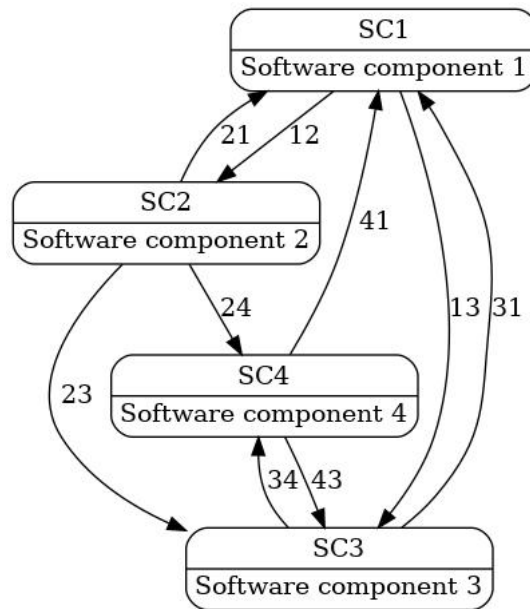
Quale delle seguenti architetture software meglio lo rappresenta ?

Scegli un'alternativa:

- ☐ 1.

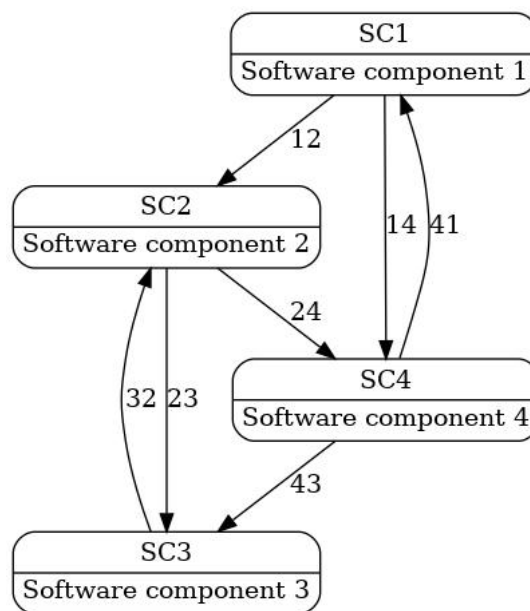


2.



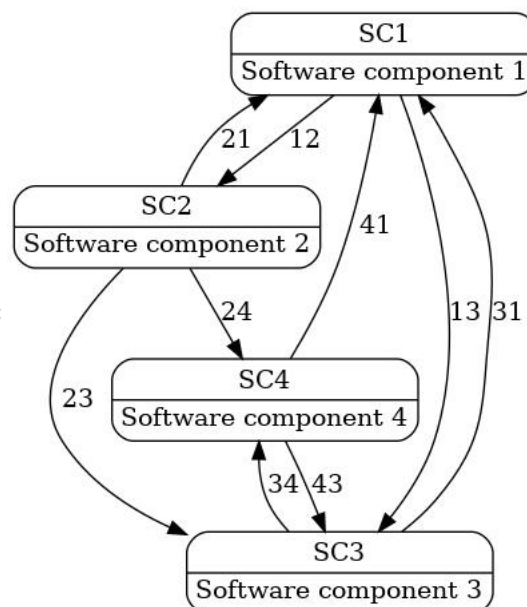
Correct.

3.



Your answer is correct.

La risposta corretta è:

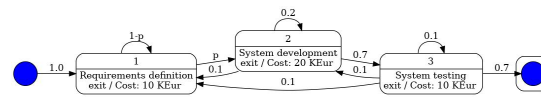


Domanda 33

Si consideri il processo software in figura

Risposta corretta

Punteggio ottenuto 1,00 su 1,00



Qual'è la migliore stima per il valore minimo di p che garantisce che la probabilità che il processo termini con un costo di esattamente 40K EUR sia almeno 0.5.

Scegli un'alternativa:

- ☐ 1. 0.785
- ☒ 2. 0.98 ✓
- ☐ 3. 0.538

Your answer is correct.

La risposta corretta è: 0.98

Domanda 34

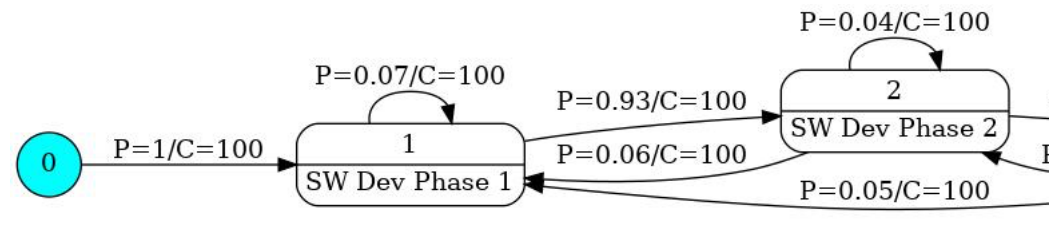
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un **Markov Decision Process**, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 400.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.880835
- ☐ 2. 0.533169
- ☒ 3. 0.76167 Correct.

Your answer is correct.

La risposta corretta è: 0.76167

Domanda 35

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale dei seguenti è un requisito non funzionale?

Scegli un'alternativa:

- ☐ 1. L'output della funzione è sempre positivo.
- ☐ 2. L'output della funzione è sempre un numero paro.
- ☒ 3. Il tempo di completamento della funzione è al più 2ms. Correct.

Your answer is correct.

La risposta corretta è: Il tempo di completamento della funzione è al più 2ms.

Domanda 36

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive l'obiettivo del "check di realismo" (realizability) che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Assicurarsi che le funzionalità richieste al sistema siano necessarie per soddisfare le necessità del customer.
- ☒ 2. Assicurarsi che, tenendo conto della tecnologia, budget e tempo disponibili, sia possibile realizzare un sistema che soddisfa i requisiti. 
- ☐ 3. Assicurarsi che le performance richieste al sistema siano necessarie per soddisfare le necessità del customer.

Your answer is correct.

La risposta corretta è: Assicurarsi che, tenendo conto della tecnologia, budget e tempo disponibili, sia possibile realizzare un sistema che soddisfa i requisiti.

Domanda 37

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive l'obiettivo del "check di completezza" che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Assicurarsi che i requisiti funzionali descrivano tutte le funzionalità del sistema.
- ☒ 2. Assicurarsi che i requisiti descrivano tutte le funzionalità e vincoli (e.g., security, performance) del sistema desiderato dal customer. 
- ☐ 3. Assicurarsi che per ogni requisito sia stato implementato nel sistema.

Your answer is correct.

La risposta corretta è: Assicurarsi che i requisiti descrivano tutte le funzionalità e vincoli (e.g., security, performance) del sistema desiderato dal customer.

Domanda 38

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quali delle seguenti attività è parte del processo di validazione dei requisiti ?

Scegli un'alternativa:

- ☒ 1. Accertarsi che i requisiti definiscano un sistema che risolve il problema che l'utente pianifica di risolvere. ✓
- ☐ 2. Accertarsi che il sistema soddisfi i requisiti dati.
- ☐ 3. Accertarsi che l'architettura del sistema soddisfi i requisiti dati.

Your answer is correct.

La risposta corretta è: Accertarsi che i requisiti definiscano un sistema che risolve il problema che l'utente pianifica di risolvere.

Domanda 39

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile x1 rappresenta lo stato precedente ad x e la variabile x2 lo stato precedente ad x1.**Si consideri il seguente requisito:****Se lo stato vale 0 allora dopo al più 2 transizioni lo stato prende il valore 1.****Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?**

- ☐ a. `assert( (x == 0) ? (x1 == 1) || (x2 == 1) : 1 )`
- ☐ b. `assert( (x2 == 0) ? (x1 == 1) || (x == 1) : 0 )`
- ☒ c. `assert( (x2 == 0) ? (x1 == 1) || (x == 1) : 1 )` ✓

Your answer is correct.

La risposta corretta è:
`assert( (x2 == 0) ? (x1 == 1) || (x == 1) : 1 )`

Domanda 40

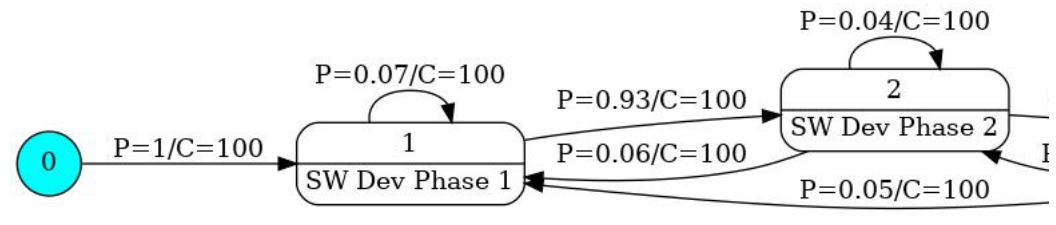
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un **Markov Decision Process**, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 500.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.5477
- ☒ 2. 0.0954 ✔ Correct.
- ☐ 3. 0.06678

Your answer is correct.

La risposta corretta è: 0.0954

Domanda 41

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il team di sviluppo di un'azienda consiste di un senior software engineer e due sviluppatori junior. Usando un approccio plan-driven (ad esempio, water-fall) la fase di design impegna solo il membro senior per tre mesi e la fase di sviluppo e testing solo i due membri junior per tre mesi. Si assuma che non ci siano "change requests" e che il membro senior costi A Eur/mese ed i membri junior B Eur/mese. Qual'è il costo dello sviluppo usando un approccio plan-driven come sopra ?

Scegli un'alternativa:

- ☐ 1. $A + 2*B$
- ☐ 2. $3*A + 3*B$
- ☒ 3. $3*A + 6*B$ ✔

Your answer is correct.

La risposta corretta è: $3*A + 6*B$

Domanda 42

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile x_1 rappresenta lo stato precedente ad x e la variabile x_2 lo stato precedente ad x_1 .

Si consideri il seguente requisito:

Se il sistema è nello stato 0 allora dopo al più due transizioni sarà nello stato 2.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☒ a. `assert( (x2==0) ? (x==2) : 1 )` ✓
- ☐ b. `assert( (x==0) ? (x2==2) : 1 )`
- ☐ c. `assert( (x==0) ? (x2==2) : 0 )`

Your answer is correct.

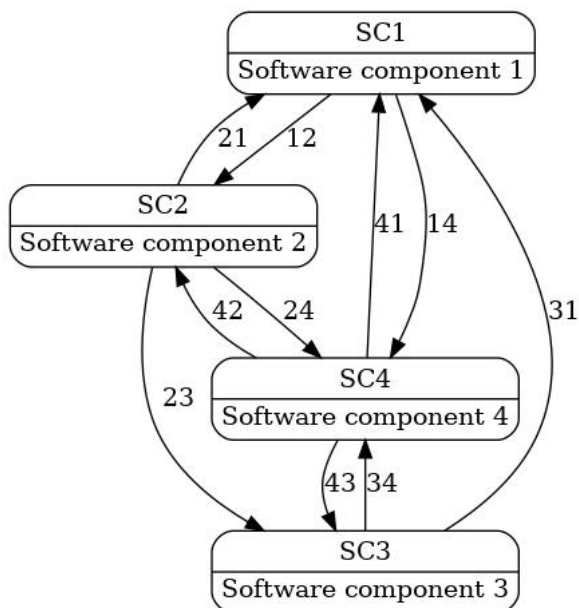
La risposta corretta è:
`assert( (x2==0) ? (x==2) : 1 )`

Domanda 43

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri la seguente architettura software:



Quale dei seguenti frammenti di pseudo codice meglio la rappresenta?

Scegli un'alternativa:

- ☐ 1. // Begin input/output connections for system components
Input 13 of SC3 = Output 13 of SC1
Input 14 of SC4 = Output 14 of SC1
Input 23 of SC3 = Output 23 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 34 of SC4 = Output 34 of SC3
Input 42 of SC2 = Output 42 of SC4
Input 43 of SC3 = Output 43 of SC4
// End input/output connections for system components
- ☒ 2. // Begin input/output connections for system components  Correct.
Input 12 of SC2 = Output 12 of SC1
Input 14 of SC4 = Output 14 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 23 of SC3 = Output 23 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 34 of SC4 = Output 34 of SC3
Input 41 of SC1 = Output 41 of SC4
Input 42 of SC2 = Output 42 of SC4
Input 43 of SC3 = Output 43 of SC4
// End input/output connections for system components
- ☐ 3. // Begin input/output connections for system components
Input 12 of SC2 = Output 12 of SC1
Input 13 of SC3 = Output 13 of SC1
Input 14 of SC4 = Output 14 of SC1
Input 23 of SC3 = Output 23 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 32 of SC2 = Output 32 of SC3
Input 34 of SC4 = Output 34 of SC3
// End input/output connections for system components

Your answer is correct.

La risposta corretta è: // Begin input/output connections for system components
Input 12 of SC2 = Output 12 of SC1
Input 14 of SC4 = Output 14 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 23 of SC3 = Output 23 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 34 of SC4 = Output 34 of SC3
Input 41 of SC1 = Output 41 of SC4
Input 42 of SC2 = Output 42 of SC4
Input 43 of SC3 = Output 43 of SC4
// End input/output connections for system components

Domanda 44

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Una azienda vende software utilizzando un contratto di Service Level Agreement (SLA) per cui l'utente paga 1000 Eur al mese di licenza e l'azienda garantisce che il software sia "up and running". Questo vuol dire che failures del software generano un costo (quello del repair). Sia $C = 10000$ Eur il costo del repair di una failure e $R = P \cdot C$ il valore atteso (rischio) del costo dovuto alle failures (dove P è la probabilità di una software failure). Ovviamente affinché il business sia profittevole deve essere che R sia al più 1000 Eur. Qual'e' il valore massimo di P che garantisce la validità del modello di business di cui sopra ?

Scegli un'alternativa:

- ☐ 1. $P = 1/10000$
- ☐ 2. $P = 1/1000$
- ☒ 3. $P = 1/10$ ✓

Your answer is correct.

La risposta corretta è: $P = 1/10$

Domanda 45

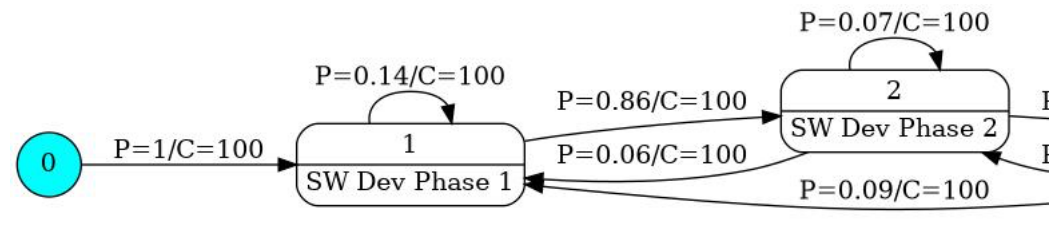
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 400.000000 ?

Scegli un'alternativa:

- ☒ 1. 0.546186 ✓ Correct.
- ☐ 2. 0.38233
- ☐ 3. 0.773093

Your answer is correct.

La risposta corretta è: 0.546186

Domanda 46

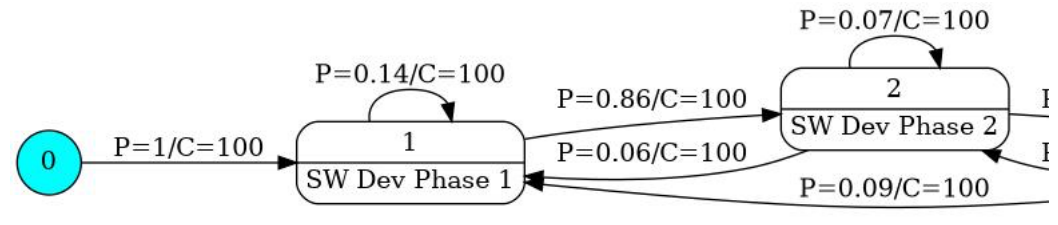
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un **Markov Decision Process**, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 500.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.5805
- ☒ 2. 0.161 ✔ Correct.
- ☐ 3. 0.1127

Your answer is correct.

La risposta corretta è: 0.161

Domanda 47

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo di sviluppo plan-driven consiste di 2 fasi F1, F2, ciascuna costo A. Alla fine di ogni fase vengono prese in considerazione le "change requests" e, se ve ne sono, lo sviluppo viene ripetuto a partire dalla prima iterazione. Quindi con nessuna change request si hanno le fasi: F1, F2 e costo 2A. Con una "change request" dopo la prima fase si ha: F1, F1, F2 e costo 3A. Con una change request dopo la fase 2 si ha: F1, F2, F1, F2 e costo 4A. Qual'è il costo nel caso in cui ci siano change requests sia dopo la fase 1 che dopo la fase 2.

Scegli un'alternativa:

- ☒ 1. 5*A ✔
- ☐ 2. 6*A
- ☐ 3. 7*A

Your answer is correct.

Nel caso in cui ci siano change requests sia dopo la fase 1 che dopo la fase 2 si ha: F1, F1, F2, F1, F2. Il costo quindi è 5*A.

La risposta corretta è: 5*A

Domanda 48

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Focalizzandosi sui metodi agile di sviluppo del software, quale delle seguenti affermazioni è vera?

Scegli un'alternativa:

- ☒ 1. Le attività di definizione dei requisiti e di sviluppo sono interleaved. 
- ☐ 2. Per evitare di sprecare tempo durante la fase di sviluppo del software, il customer non è mai coinvolto nel processo di sviluppo del software.
- ☐ 3. Per evitare di sprecare tempo durante la fase di sviluppo del software, questa inizia solo quando i requisiti sono stati completamente definiti.

Your answer is correct.

La risposta corretta è: Le attività di definizione dei requisiti e di sviluppo sono interleaved.

Domanda 49

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri la seguente funzione di transizione.

```
#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (4);
        case 1: return (2);
        case 2: return (0);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (0);
        case 1: return (2);
        case 2: return (2);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (1);
        case 1: return (4);
        case 2: return (4);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (1);
        case 1: return (2);
        case 2: return (3);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (3);
        case 1: return (1);
        case 2: return (1);
        default: return (0); }
    default: return (0);
  }
} // f()
```

Quale dei seguenti UML state diagram rappresenta correttamente la macchina a stati finiti definita dalla funzione di transizione definita sopra?